



CWPP BEST PRACTICES CHEAT SHEET

Stop the attack inside the workload

Six runtime security practices for containers, Kubernetes and VMs, from the first baseline to Zero Trust enforcement in the kernel.

Runtime attacks finish before the alert lands

Attackers automate every step from finding a weak workload to running code inside it. A tool that kills a process after the alert sees the miner, the shell or the data theft after it already ran.

10 min

Average time from reconnaissance to a completed cloud attack.

Sysdig 2023 Global Cloud Threat Report. sysdig.com

45%

Share of organizations that had a runtime incident in the last 12 months.

Red Hat State of Kubernetes Security, 2024. redhat.com

\$53

Cloud bill a cryptojacking victim pays for every \$1 the attacker mines.

Sysdig 2022 Cloud Native Threat Report. sysdig.com

Four gaps let a runtime attack **run to the end**



Detect, then react

A post attack tool kills the process after the alert fires. By then the attacker binary ran and could disable controls, delete logs or send data out.



No baseline of normal

Nobody recorded which processes, files and connections each container uses. So a cronjob and an attack look the same, and every rule stays in audit.



Evidence leaves with the pod

Attackers stay a median of 11 days before discovery. Yet 60% of containers live for 60 seconds or less, so the trail is gone when the hunt starts.



A flat cluster network

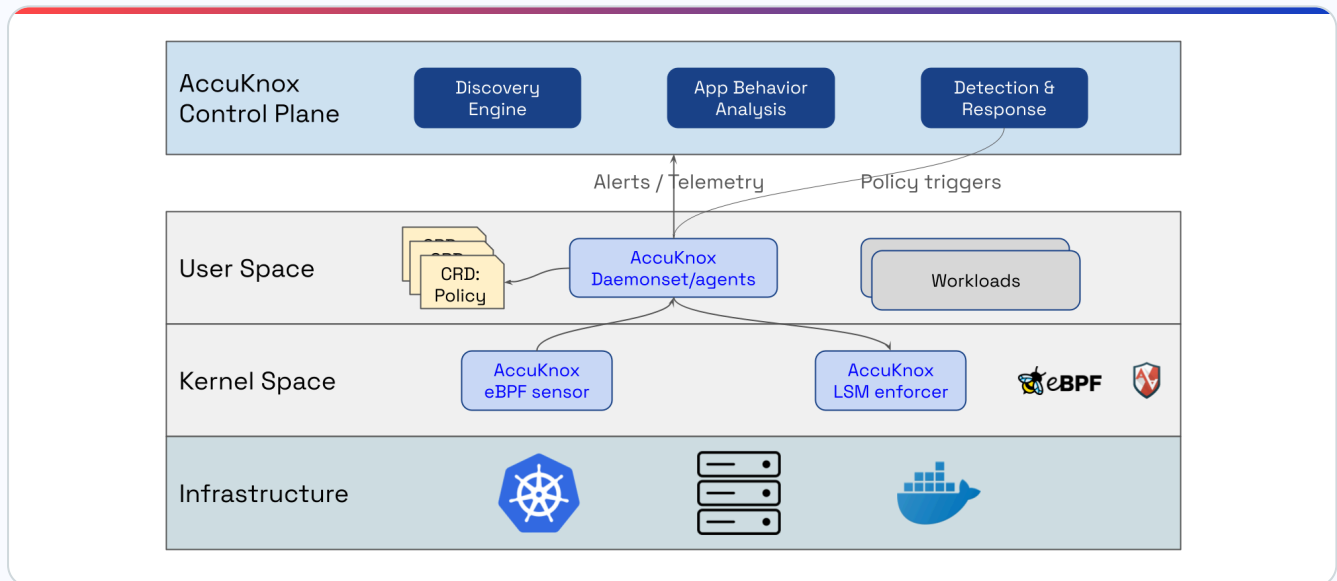
Kubernetes lets every pod reach every other pod by default. One compromised pod becomes the path to the database.

Sources. Dwell time from the Mandiant 2025 annual threat report, cloud.google.com. Container lifespan from the Sysdig 2025 Cloud Native Security and Usage Report, sysdig.com.

Start here

Six practices move you from audit to block

AccuKnox runs an eBPF sensor and an LSM enforcer on every node, through the open source KubeArmor. The six practices follow the Runtime Security Journey: learn, harden, enforce, then keep the evidence.



Runtime security architecture. help.accuknox.com/getting-started/runtime-sec-arch

01	Learn app behavior Baseline each workload first	05	04	Segment the network Stop lateral movement between pods	08
02	Apply hardening policies Start from CIS, MITRE and NIST	06	05	Stop miners, lock files Cryptojacking and FIM	09
03	Enforce Zero Trust inline Block in the kernel	07	06	Keep forensics Evidence that outlives the pod	10

Learn what each workload **does** first

The Discovery Engine records every process, file and network call per container, then writes that behavior as a policy you review.

The screenshot shows the AccuKnox App Behavior interface. On the left is a sidebar with navigation links: Dashboard, Inventory, Issues, Compliance, Runtime Protection, CWPP Dashboard, App Behavior (selected), Policies, and Targets. The main panel is titled 'App Behavior' and has filters for 'azure-staging', 'wordpress...', and 'mysql-6c6...'. It includes tabs for 'File Observability', 'Process Observability' (selected), and 'Network Observability'. A table displays process observability data with columns: Last Update, Process, File Path Accessed, Container, Occurance, Status, and Details. A purple arrow points to the 'Status' column.

Last Update	Process	File Path Accessed	Container	Occurance	Status	Details
02/17/2023 08:47 AM	/bin/dash	/usr/sbin/mysqld	mysql	1	Allow	Details
02/17/2023 08:47 AM	/bin/bash	/bin/date	mysql	7	Allow	Details
02/17/2023 09:52 AM	/bin/bash	/bin/lis	mysql	8	Allow	Details
02/17/2023 08:47 AM	/bin/bash	/usr/sbin/mysqld	mysql	1	Allow	Details
02/17/2023 08:47 AM	/usr/bin/containerd-shim-runc-	/usr/sbin/mysqld	mysql	1	Allow	Details

App Behavior, process observability. help.accuknox.com/use-cases/app-behavior/

DO

- Onboard each cluster and VM, then let discovery watch live traffic.
- Review the File, Process and Network views for each workload in App Behavior.
- Accept or discard each new discovered policy as cronjobs and code paths appear.
- Keep policies in AUDIT for 2 to 3 weeks, until behavior holds steady and the policy is STABLE.

DON'T

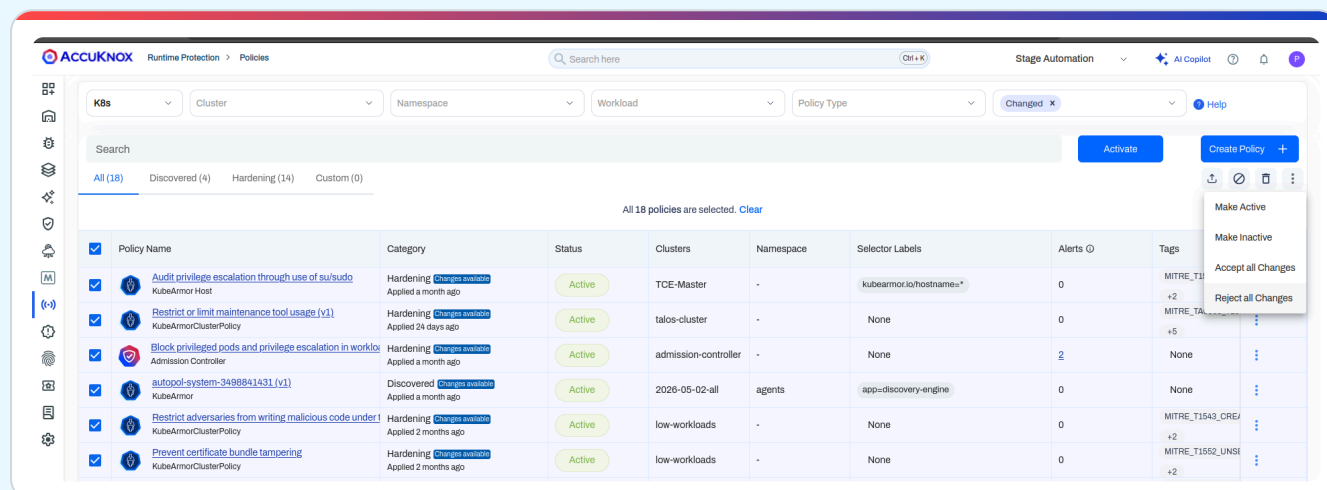
- Don't write allow lists by hand from a design document.
- Don't move a policy to BLOCK before it reaches STABLE.
- Don't stop discovery after go live. Step 5 of the journey loops back to step 2.

HOW ACCUKNOX CLOSSES THE GAP

Discovery never stops, so the golden baseline keeps tracking real behavior for every container.
[Read the App Behavior guide](#)

Harden workloads with framework policies

KubeArmor ships hardening policies that map to CIS, MITRE ATT&CK, NIST, PCI DSS and STIG controls.



Runtime Protection policies with hardening tags. help.accuknox.com/use-cases/app-hardening/

DO

- Deny package managers such as apt, apk and yum in production containers.
- Block execution of any binary from **/tmp/**.
- Protect the service account token when no process in the pod uses it.
- Drop capabilities the app never uses, such as NET_RAW.

DON'T

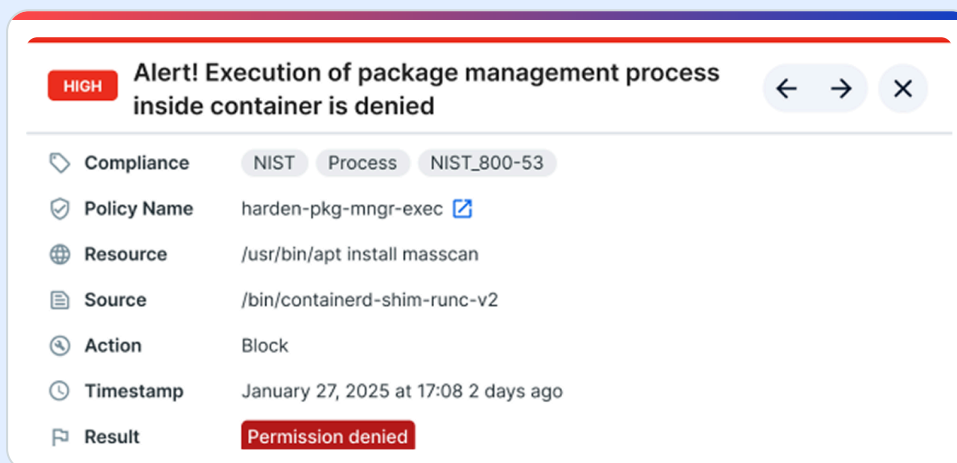
- Don't leave admin, discovery and scan tools runnable in production pods.
- Don't apply every hardening policy in BLOCK on day one. Run AUDIT first.
- Don't treat an image scan as hardening. A clean image can still run an attacker binary.

HOW ACCUKNOX CLOSES THE GAP

Pick a cluster and namespace in Runtime Protection, select the hardening policies, then apply and approve them. [Browse the hardening policies](#)

Block in the kernel, not after the alert

KubeArmor enforces through LSMs such as AppArmor and BPF LSM, so a denied process never starts.



A blocked package manager run, result Permission denied. help.accuknox.com/use-cases/zero-trust/

DO

- Move STABLE policies to BLOCK, so known behavior runs and the rest is denied.
- Allow list the processes each container may run.
- Allow network access only for the binaries that need it.
- Let only named processes read credential and config files.

DON'T

- Don't rely on killing a process after an alert. The binary already ran.
- Don't count on signatures. A deny by default policy rejects unknown malware too.
- Don't expect enforcement on nodes without eBPF and LSM support. They drop to visibility only.

HOW ACCUKNOX CLOSSES THE GAP

Enforcement runs on each node with no sidecar and no inline proxy, and keeps working if the control plane is unreachable. [Read the Zero Trust guide](#)

Segment pod traffic to stop lateral movement

Every pod can reach every other pod by default. A network policy limits each pod to the flows it needs.

The screenshot shows the 'Policies' page in the AccuKnox console. It includes filters for Workloads (K8s), Cluster (Cluster(1)), Namespace (Namespace(1)), and Policy Type (Policy Type(1)). A search bar and an 'Apply' button are at the top right. Below the filters, a table lists discovered policies. The table has columns for Policy Name, Category, Status, Clusters, Namespace, Selector Labels, and Tags. Four policies are listed, all with a status of 'Inactive'.

Policy Name	Category	Status	Clusters	Namespace	Selector Labels	Tags
autopol-ingress-olgmilncujzbtq Kubernetes Network	Discovered	Inactive	azure-staging	wordpress-mysql	app=mysql...	None
autopol-ingress-ylkpyzmtbhlqpryg Kubernetes Network	Discovered	Inactive	azure-staging	wordpress-mysql	app=wordpress...	None
autopol-egress-heedbajpwwshxmo Kubernetes Network	Discovered	Inactive	azure-staging	wordpress-mysql	app=mysql...	None
autopol-egress-lzuioxmzgdfoajx Kubernetes Network	Discovered	Inactive	azure-staging	wordpress-mysql	app=wordpress...	None

Discovered network policies for a sample app. help.accuknox.com/use-cases/network-segmentation/

DO

- Review the discovered ingress and egress policies for each app.
- Segment by application tier, such as web to database only.
- Segment by compliance scope, so regulated pods sit apart.
- Approve each applied policy, because it waits in Pending until you do.

DON'T

- Don't treat a namespace as a network boundary. Pods on one node share Layer 2.
- Don't write network rules before you see real traffic in App Behavior.
- Don't let scan tools in a pod use ICMP to map the network.

HOW ACCUKNOX CLOSES THE GAP

AccuKnox segments at the pod level and moves each policy from Pending to Active after approval. [Read the segmentation guide](#)

Stop miners and lock system files

Of 50 compromised GCP instances, 86% ran cryptocurrency miners. A miner must download, write and run a binary, so block each step.

The screenshot shows the AccuKnox K8s interface. On the left is a sidebar with navigation links: Dashboard, Inventory, Issues, Compliance, Runtime Protection, Remediation, Monitors / Alerts (selected), Triggers, and Monitors. The main panel displays a list of alerts under the 'Alerts' tab. A critical alert is highlighted, titled 'Detected and prevented compromise to File integrity'. The alert details include:

- Policy Name:** harden-file-integ_
- Resource:** /usr/sbin/file
- Source:** /usr/bin/touch file
- Action:** Block
- Result:** Permission denied
- Severity:** 1

Below the alert details is a table with columns: Operation, Cluster Name, Pod Name, Workload Name, and Workload Type. The data row shows: Operation: File, Cluster Name: VM-host, Pod Name: --, Workload Name: master-node, Workload Type: Node.

At the bottom, there is a 'Raw Logs' section with a 'Copy' button. The raw log content is:

```
{
  "Action": "Block"
  "ClusterName": "VM-host"
  "Cwd": "/usr/sbin/"
}
```

A blocked write to a system folder. help.accuknox.com/use-cases/vm-file-integrity/ Miner data from Google Cloud Threat Horizons, November 2021, services.google.com

DO

- Block known miners such as xmrig and dero by name.
- Block masscan, zgrab2, nmap and ntpdate, which miners use for recon and time sync.
- Make **/bin/**, **/sbin/**, **/usr/bin/** and **/boot/** read only.
- Cover config and credential paths, which PCI DSS, NIST and CIS expect FIM to watch.

DON'T

- Don't wait for an OOMKilled pod to tell you a miner runs.
- Don't rely on a miner name list alone. Zero Trust policies also stop the next unknown miner.
- Don't only alert on writes to system folders. Block them.

HOW ACCUKNOX CLOSES THE GAP

In the documented test the miner exits with status 126, and the alert shows Permission denied. [Read the cryptojacking guide](#)

Keep evidence that outlives the container

60% of containers live for 60 seconds or less. Capture process, file, network and syscall events before the pod is gone.



Runtime alerts timeline. help.accuknox.com/use-cases/forensics/ Lifespan data from Sysdig, 2025, sysdig.com

DO

- Audit process, file, network and syscall events on critical workloads.
- Audit every access to **/etc/shadow**, **/etc/sudoers** and **/etc/pam.d/**.
- Block log deletion on critical components.
- Forward alerts to Splunk, ELK or Rsyslog, and send high severity alerts to Slack.

DON'T

- Don't rely on node logs alone. They lose the pod and container context.
- Don't ignore process enumeration with `ps`, `pgrep` or `top`.
- Don't let alerts stay in the console. Route each one to a channel with an owner.

HOW ACCUKNOX CLOSES THE GAP

Each alert records the cluster, pod, container, image, process, parent process, enforcer and result. [Read the forensics guide](#)

Find your gap, then take the first step

Read each row. Where the gap is true for you, take the first step this week.

IF THIS IS TRUE	ACCUKNOX CAPABILITY	FIRST STEP
Nobody can list what each container runs, reads and connects to	App Behavior and Discovery Engine	Onboard one cluster and review App Behavior for one namespace
Production pods can run apt, curl downloads or binaries from /tmp/	Hardening policies	Apply the package manager and /tmp/ noexec policies in AUDIT
Your runtime tool kills processes after an alert	KubeArmor inline enforcement	Move one STABLE policy from AUDIT to BLOCK
Any pod can reach any other pod	Network segmentation	Review and approve the discovered network policies for one app
A miner shows up only as a cloud bill or an OOMKilled pod	Cryptojacking and FIM policies	Apply the miner policy and make system binary folders read only
Evidence of an incident leaves with the pod	Forensics and alert forwarding	Audit sensitive files and forward alerts to your SIEM

See inline runtime protection on your cluster

Bring your gap check to a demo. An AccuKnox engineer will show discovery, hardening and BLOCK mode on a live workload.

[Book a demo](#)[Open the CWPP docs](#)

KuppingerCole 2026 CNAPP Leadership Compass highlights the AccuKnox eBPF based runtime enforcement.

accuknox.com/analyst-recognition

Analyst Andrew Green placed AccuKnox in the Runtime Environment group for kernel level enforcement.

accuknox.com/analyst-recognition

KubeArmor, created by AccuKnox, performs inline mitigation against zero day attacks.

accuknox.com/platform/cwpp